Go 语言单元测试实验报告

1. 实验目的

本实验的目标如下：

- 选择一个实际开源库中的工具方法作为待测对象，理解其功能与接口行为
- 将待测库源码本地化到实验工程中，便于修改与测试
- 人为注入一个不影响编译但会影响功能正确性的缺陷
- 分别使用手工测试方式和 Go 标准测试框架对目标代码进行验证
- 对比两种测试方式在可维护性、可复用性和自动化能力上的差异

2. 实验环境

- 编程语言：Go
- 测试对象库： `github.com/spf13/cast`
- 测试框架：Go 标准库 `testing`

3. 测试对象说明

本次实验选择的测试对象是 Go 语言第三方库 `github.com/spf13/cast` 。

`cast` 是一个常用的类型转换工具库，主要作用是将不同类型的数据转换为目标类型，例如：

- `string` 转 `int`
- `string` 转 `bool`
- `int` 转 `string`
- `nil` 转默认零值

该库适用于配置解析、输入转换、接口数据处理等场景。由于其输入类型多样、转换规则明确，因此非常适合作为单元测试实验对象。

本实验重点选择了以下两个关键方法作为主要测试对象：

- `ToIntE` ：将输入值转换为 `int` ，如果失败则返回错误
- `ToBoolE` ：将输入值转换为 `bool` ，如果失败则返回错误

同时，为了演示多个测试文件可以被统一批量执行，本实验还补充测试了一个未注入缺陷的方法：

```
case string:
        // 为实验注入的缺陷：合法字符串 "false"
        // 会被错误地当成 true 处理。
        if b == "false" {
                return true, nil
        }
```

- `ToStringE` ：将输入值转换为 `string`
- 输入输出关系清晰，便于构造断言
- 既有正常输入，也有非法输入，适合做边界分析
- 容易注入逻辑错误，并观察测试是否能够发现问题

4. 工程结构与源码替换方式

为了便于分析和修改源码，本实验将 `cast` 库的源码克隆到当前项目中的 `cast/` 目录下，并通过 `replace` 将依赖替换为本地版本。

当前工程关键结构如下：

```
my-go/
├── cast/
│   └── ... cast 本地源码
├── go.mod
├── main.go
├── main_test.go
├── string_test.go
```

`go.mod` 中的核心配置如下：

```
require github.com/spf13/cast v0.0.0

replace github.com/spf13/cast => ./cast
```

- 保留对 `github.com/spf13/cast` 的正常导入方式
- 在当前项目内优先使用 `./cast` 目录中的本地源码
- 便于直接修改第三方库代码并验证修改效果

5. 缺陷注入设计

根据软件缺陷常见类型中的“逻辑错误”和“条件判断错误”，本实验在 `cast/basic.go` 的 `ToBoolE` 方法中注入了一个功能性缺陷。

缺陷描述如下：

- 输入字符串 `"false"` 时
- 正确结果应为 `false`
- 注入缺陷后的错误结果为 `true`

缺陷位置如下：

- 文件：basic.go
- 方法： `ToBoolE`
- 缺陷类型：条件判断错误、逻辑错误

注入后的核心代码逻辑可概括为：

```go
if b == "false" {
    return true, nil
}
```

该缺陷具备以下特点：

- 不会引起编译错误
- 会造成运行结果与预期不一致
- 可以通过单元测试稳定复现

因此非常适合作为实验中的“待检测缺陷”。

6. 手工测试设计与实现

本实验首先采用不依赖测试框架的方式，在 `main.go` 中通过 `main` 方法直接驱动待测函数。

手工测试覆盖的方法如下：

- `ToIntE`
- `ToBoolE`

对应测试案例如下：

- `ToIntE("123")` ： 应返回 `123`
- `ToIntE("abc")` ： 应返回错误
- `ToBoolE("true")` ： 应返回 `true`
- `ToBoolE("false")` ： 应返回 `false`

程序运行时会逐项输出“通过”或“失败”，并在最后汇总统计结果。

运行命令如下：

```
go run main.go
```

实验结果如下：

```
[通过] ToIntE 可以解析数字字符串
[通过] ToIntE 遇到字母字符串应返回错误
[通过] ToBoolE 可以解析 true
[失败] ToBoolE 可以解析 false: 得到 true，期望 false

手工测试汇总：通过 3 项，共 4 项
```

结果分析：

- `ToIntE` 的正常与异常场景均符合预期
- `ToBoolE("true")` 行为正常
- `ToBoolE("false")` 出现错误，说明注入的缺陷被手工测试成功检测到

7. 基于 testing 的单元测试设计

在完成手工测试后，本实验进一步使用 Go 标准库 `testing` 编写规范化的单元测试，以模拟 JUnit 这类测试框架的使用方式。

7.1 测试文件划分

本实验使用了多个测试文件，以演示批量执行能力：

- `main_test.go`
  负责测试重点分析的 `ToIntE` 和 `ToBoolE`
- `string_test.go`
  负责测试 `ToStringE` ，用于展示多个 `*_test.go` 文件可统一执行

7.2 测试组织方式

`main_test.go` 中定义了测试集 `TestCastSuite`，内部再通过 `t.Run` 组织多个子测试：

- `ToIntE`
- `ToBoolE`

`string_test.go` 中定义了独立测试函数：

- `TestToStringE`

这样组织后，只要运行一次 `go test .`，当前包下多个测试文件中的所有测试都会被统一发现并批量执行。

7.3 测试案例

框架测试包含以下案例：

- `ToIntE` 可以解析合法数字字符串
- `ToIntE` 遇到非法字符串应返回错误
- `ToBoolE` 可以解析 `true` 字符串
- `ToBoolE` 可以解析 `false` 字符串
- `ToStringE` 可以将整数转换为字符串
- `ToStringE` 可以将布尔值转换为字符串
- `ToStringE` 可以将 `nil` 转换为空字符串

7.4 运行方式与结果

运行命令如下：

```
go test
```

实验结果如下：

```
--- FAIL: TestCastSuite (0.00s)
    --- FAIL: TestCastSuite/ToBoolE (0.00s)
        --- FAIL: TestCastSuite/ToBoolE/可以解析_false_字符串 (0.00s)
            main_test.go:46: 得到 true，期望 false
FAIL
FAIL    my-go-project   0.584s
FAIL
```

结果分析：

- 框架测试定位到了 `ToBoolE` 中关于 `"false"` 输入的失败案例
- 失败信息明确给出了测试名称、所在文件和断言原因
- 新增的 `string_test.go` 并未受缺陷影响，说明多个测试文件可以共存并统一执行

8. 测试案例汇总表

编号	待测方法	输入	期望结果	实际结果	说明
1	`ToIntE`	`"123"`	返回 `123`，无错误	通过	正常功能测试
2	`ToIntE`	`"abc"`	返回错误	通过	非法输入测试
3	`ToBoolE`	`"true"`	返回 `true`，无错误	通过	正常功能测试
4	`ToBoolE`	`"false"`	返回 `false`，无错误	失败	成功检测到注入缺陷
5	`ToStringE`	`123`	返回 `"123"`，无错误	通过	批量执行示范案例
6	`ToStringE`	`true`	返回 `"true"`，无错误	通过	批量执行示范案例
7	`ToStringE`	`nil`	返回空字符串，无错误	通过	批量执行示范案例

9. 两种测试方法的优缺点对比

9.1 手工测试方法的优点

- 实现直接，容易理解测试原理
- 不依赖测试框架，适合课堂演示和初步验证
- 可以快速验证少量功能点

9.2 手工测试方法的缺点

- 测试代码与业务代码混在一起，结构不够清晰
- 难以组织大量测试案例
- 批量执行能力弱，维护成本高
- 失败信息不够规范
- 不利于重复执行与回归测试

9.3 testing 框架测试的优点

- 测试代码独立于业务代码，结构更清晰
- 可以通过多个 `*_test.go` 文件组织测试
- 运行 `go test .` 即可批量执行当前包全部测试
- 失败时能准确定位到测试函数、子测试和断言位置
- 更适合长期维护和自动化集成适配CI/CD

9.4 testing 框架测试的缺点

- 需要了解测试函数命名和执行规则
- 初学时需要熟悉 `testing` 的基本写法

10. 实验结论

本实验选取 `cast` 库中的 `ToIntE` 和 `ToBoolE` 作为主要测试对象，并补充了 `ToStringE` 作为批量执行示例。通过将 `cast` 源码本地化到实验工程中，并使用 `replace` 替换依赖，实现了对第三方库源码的直接修改与测试。

实验中在 `ToBoolE` 方法中注入了一个逻辑缺陷，使字符串 `"false"` 被错误地转换为 `true` 。随后分别采用手工测试和 `testing` 框架测试两种方式进行验证，结果都成功检测到了该缺陷。

实验结论如下：

- 手工测试适合展示测试思想，但不适合复杂场景和长期维护
- 基于 `testing` 的单元测试能够更规范地组织测试代码
- 多个 `*_test.go` 文件可以统一批量执行
- 标准测试框架更适合工程实践